

Beyond Directional Accuracy: A Reproducible Audit of Daily Cryptocurrency Forecasts and Trading Costs

Abstract

Directional forecasting accuracy, incremental predictive value, and trading profitability are different research outcomes. This study examines these distinctions through a reproducible audit of a cryptocurrency forecasting implementation. Using daily Binance spot data for Bitcoin, Ether, and Dogecoin, we construct 30 causal price, volume, trading-activity, and calendar features and evaluate logistic regression and histogram gradient boosting in expanding-window tests. The evaluation yields 5562 out-of-sample asset-day predictions over 2083 distinct decision dates. Logistic regression achieves 52.95% pooled accuracy, compared with 52.88% for a parameter-free inverse-persistence rule. A date-synchronized stationary bootstrap estimates an accuracy difference of 0.07 percentage points, with a 95% interval of $[-1.44, 1.61]$. An idealized equal-weight long/short return diagnostic has gross annualized Sharpe 0.544 and net Sharpe 0.113 at an assumed nine basis points per unit of one-way turnover. Joint resampling of gross returns and turnover gives a break-even cost estimate of 11.36 basis points, with a 95% interval of $[-4.00, 26.43]$. A separate next-open-to-close cash ledger illustrates the sensitivity to mandatory daily liquidation. Regression fixtures detect eight deliberately introduced software mutations. The results do not establish incremental superiority over the reversal baseline or robust trading profitability. The contribution is an auditable case study linking prediction, uncertainty, implementation contracts, and economic evaluation, rather than a claim of a new forecasting algorithm.

Keywords: Cryptocurrency, Directional forecasting, Walk-forward evaluation, Backtesting, Transaction costs, Reproducibility

1. Introduction

An accurate classifier need not be an economically useful trading system. A directional prediction receives the same credit for correctly forecasting a small move as a large one, whereas trading profits depend on signed return magnitudes, position sizes, implementation timing, and costs. A second distinction is equally important: a classifier can exceed a 50% hit rate while offering little additional information relative to a simple observable rule. A third concerns measurement itself. A backtest may apply a model consistently yet implement the wrong execution or accounting convention.

This paper studies these issues in one supplied cryptocurrency forecasting artefact. The empirical question is narrow: do technical-feature classifiers improve next-day directional accuracy over elementary baselines on the same dates, and what economic conclusions remain under explicit cost and execution assumptions? The engineering question is whether executable tests detect selected faults that could compromise the interpretation of the re-

sult. These are separate questions; neither statistical significance nor passing software tests answers both.

The study makes three contributions. First, it provides a complete prediction-level replication with a reconciled sample, fold-specific preprocessing, saved fitted pipelines, and a feature inventory tied to executable code. Second, it measures model–baseline differences with serial-dependence-aware per-asset inference and synchronized date-block resampling, and estimates the cost root directly from jointly resampled returns and turnover. Third, it separates an idealized target-notional return diagnostic from a fully liquidated daily cash-ledger experiment, while documenting the scope of regression and mutation tests.

The main finding is not that cryptocurrency cannot be forecast. It is that the additional value of the tested classifiers over inverse persistence is not established in this sample. The pooled accuracy difference is small relative to its uncertainty. The financial result is also sensitive to costs and the trading policy translating forecasts into positions. These findings support a reproducibility and evaluation case study, not a universally profitable strategy or a claim about all cryptocurrency models.

2. Related research and positioning

Research on cryptocurrency predictability provides mixed findings across horizons, inputs, and model classes. Jaquart et al. (2021), in this journal, examine Bitcoin prediction horizons from one to sixty minutes and compare several machine-learning models. Their setting differs from this paper’s daily, three-asset study. Gradojevic et al. (2023) consider technical predictors at hourly and daily frequencies and report that forecasting results depend on horizon and specification. These studies motivate explicit statements of sampling frequency, benchmarks, and input information; they do not imply that their reported performance should transfer to the present data or implementation.

Arnott et al. (2019) discuss disciplined financial backtesting in the machine-learning setting. Our contribution is an executable, single-artefact application of that general research concern. We do not introduce a new model family or establish the prevalence of software defects across published research. The provenance problem is related to the formulation of data leakage in Kaufman et al. (2012): information must be assessed relative to its availability for the prediction task, rather than inferred from a variable’s name.

For inference, the paired forecast-comparison perspective of Diebold and Mariano (1995) motivates analyzing observation-level loss differences. We use a Bartlett-kernel heteroskedasticity and autocorrelation consistent (HAC) estimate following Newey and West (1987). The cross-asset analysis uses the stationary bootstrap of Politis and Romano (1994), with a single resampled date index applied to all asset columns. Finally, mutation testing, reviewed by Jia and Harman (2011), evaluates whether selected erroneous implementations are detected by tests. It provides conditional evidence about those mutations, not proof that the entire system is correct.

3. Research artefact and data provenance

3.1. Scope of the implemented model

The empirical implementation is a daily technical-feature forecasting pipeline with logistic and boosted-tree classifiers. An additional one-feature logistic model is a diagnostic

ablation. The broader software repository contains components for other forecasting and text-processing workflows; those components are not trained or validated in this experiment. In particular, the reported results are not evidence for an LLM, FinBERT, Prophet, recurrent-network ensemble, live exchange service, or autonomous trading deployment.

The experiment is retrospective. The initial model settings are inherited from the supplied evaluation source; no new search for an attractive test-period configuration is performed. However, no preregistration or untouched final holdout is available, and the comparisons and audit checks were developed with knowledge of the existing artefact. Consequently, the results are replication and exploratory evidence, not prospective confirmation of a selected strategy.

3.2. Market data and sample reconciliation

The inputs are frozen daily Binance spot klines for BTCUSDT, ETHUSDT, and DOGEUSDT. The stored fields include open, high, low, close, base volume, quote volume, trade count, and taker-buy base volume. The exchange documents the kline format and public archive (Binance, 2026). Bar dates refer to UTC opening dates; the entire day’s bar is available only after that day ends.

BTC and ETH raw histories run from 17 August 2017 through 1 April 2026, while DOGE begins on 5 July 2019. The validation checks find no duplicate or missing daily dates in these files, and enforce positive prices and consistent OHLC bounds. SHA-256 hashes identify the exact supplied inputs. An additional read-only check compares eight numeric fields for 275 overlapping asset-days in July 2019, January 2023, and March 2026 with the exchange’s checksum-verified monthly archives. All compared fields match within relative tolerance 10^{-10} and absolute tolerance 10^{-8} . This is a selected-month verification, not an independent audit of every historical observation.

Table 1: Sample reconciliation. Every asset has a final decision date of 30 March 2026. The final realized label date is 31 March 2026.

Asset	Raw rows	Usable	OOS	Folds	First OOS decision
BTC	3,150	3,088	2,083	9	2020-07-17
ETH	3,150	3,088	2,083	9	2020-07-17
DOGE	2,463	2,401	1,396	6	2022-06-04

Feature warm-up removes the first 60 rows. The final two raw bars are excluded as prediction origins to preserve the supplied study’s cutoff: one supports the final included label, and the last is an unused buffer. This explains why the usable count is raw rows minus 62, although the classification horizon itself is only one day. Initial training and the five-row gap then account for 1,005 usable rows per asset before evaluation begins (Table 1). There are 5562 asset-day forecasts but only 2083 distinct decision dates. These counts must not be interchanged when discussing dependence or effective information.

3.3. Why the sentiment proxy is excluded

The supplied sentiment-generation script explicitly describes a simulation based on a market proxy. It constructs an intraday return $u_t = (C_t - O_t)/O_t$, then a score

$$s_t = \text{round}_3[\text{clip}(\text{clip}(15u_t, -1, 1) + \epsilon_t, -1, 1)], \quad \epsilon_t \sim U(-0.2, 0.2). \quad (1)$$

Its associated headline is selected from fixed templates according to the score. This is not an independent historical news source, and the original generator does not fix the random seed. It is therefore excluded from all model inputs and results in this paper.

Using this score to predict the same bar before the bar closes would violate the information boundary. Using it after the close to predict a later return is not automatically look-ahead leakage; it is a noisy transformation of already-observed price information. A high contemporaneous correlation alone cannot determine which problem applies. The appropriate audit records construction, source, availability time, and target alignment. We do not infer misconduct from a generator that already labels itself a simulation, or make claims about genuine text-based sentiment models.

4. Methods

4.1. Features, target, and information boundary

Let $\mathcal{F}_t$ contain complete market bars through the close of day t . Each feature vector x_t is a function of $\mathcal{F}_t$. The binary target and associated simple return are

$$y_t = \mathbf{1}(C_{t+1} > C_t), \quad r_{t+1} = C_{t+1}/C_t - 1. \quad (2)$$

A tie belongs to class zero. Binary labels do not imply a 50% unconditional up probability; therefore, a coin flip is not the only relevant benchmark. The executable feature matrix contains 30 columns, not the 34 stated in the supplied draft. Appendix A gives all definitions. No full-sample imputation, cross-asset predictors, sentiment proxy, or forward-return variable enters training.

4.2. Expanding-window evaluation and model fitting

For each asset, index usable observations by $i = 0, \dots, n - 1$. Fold starts are $a_k = 1005 + 250k$. Training uses $i < a_k - 5$, and testing uses $a_k \leq i < \min(a_k + 250, n)$. A final test block is retained only if it contains at least 30 observations. The earliest fold has 1,000 training rows; later training sets expand. Every reported observation is predicted exactly once out of sample.

A standard scaler is fitted separately on each training fold and remains unchanged during that fold’s prediction. Both the fitted scaler and classifier are saved together. The implementation exports the last training-feature date, last training-label date, and test dates, so the information gap can be checked directly. Five excluded rows provide separation for the one-day labels. This is not a claim that all historical feature windows are disjoint: shared past observations in causal rolling features do not by themselves constitute future-information leakage.

The logistic model uses ℓ_2 regularization with inverse penalty parameter $C = 0.1$, an intercept, and a maximum of 2,000 iterations. Histogram gradient boosting uses maximum depth three, 200 boosting iterations, learning rate 0.05, ℓ_2 leaf regularization one, no early stopping, and random seed zero. These settings are fixed across assets and folds. Predictions use probability greater than 0.5 for class one, with ties assigned to zero. A lag-1-only logistic model uses the same penalty and fitting protocol, but retains only $\log(C_t/C_{t-1})$. It is reported as an ablation rather than selected as the preferred strategy.

4.3. Baselines and estimands

Persistence predicts yesterday's realized close-to-close direction, $\hat{y}_t^P = \mathbf{1}(C_t > C_{t-1})$. Inverse persistence predicts $1 - \hat{y}_t^P$. Both require no fitted parameter and are evaluated on exactly the model's dates. The majority baseline predicts the more frequent class in the current training fold, with ties assigned to zero. A long-only basket is a separate return benchmark, not a directional classifier.

For a model M and inverse-persistence baseline I , the primary per-observation comparison is

$$d_{j,t} = \mathbf{1}(\hat{y}_{j,t}^M = y_{j,t}) - \mathbf{1}(\hat{y}_{j,t}^I = y_{j,t}). \quad (3)$$

We report each asset's mean difference and confidence interval. A positive difference favors the model. We also report pooled accuracy as the descriptive fraction of correctly classified asset-days. Its paired difference is observation-weighted, $\hat{\Delta} = \sum_{j,t} d_{j,t}/N$, rather than an equal-weight average of three asset accuracies.

4.4. Dependence-aware uncertainty

For a single asset, let n be its OOS count and $v_t = d_t - \bar{d}$. The HAC variance of the mean is

$$\widehat{\text{Var}}(\bar{d}) = \frac{1}{n} \left[\hat{\gamma}_0 + 2 \sum_{\ell=1}^K \left(1 - \frac{\ell}{K+1} \right) \hat{\gamma}_\ell \right], \quad \hat{\gamma}_\ell = \frac{1}{n} \sum_{t=\ell+1}^n v_t v_{t-\ell}, \quad K = 20. \quad (4)$$

Intervals use the normal approximation and two-sided p -values. These are approximate dependent-data comparisons, not exact small-sample tests. The zero-variance, nonzero-difference case is explicitly marked undefined rather than assigned a nonsignificant p -value.

For pooled inference and return statistics, we use 2,000 circular stationary-bootstrap replicates with mean block length 20 dates and fixed seed 20260401. At each step, an index continues to the next observed date with probability $1 - 1/L$, or restarts at a uniformly sampled date with probability $1/L$. The same sampled date is applied to all assets, including the missing-asset mask. The pooled numerator and denominator are recomputed in every replicate. This preserves contemporaneous dependence within the resampled panel; pooling asset-days as independent Bernoulli trials would not.

We report percentile 95% intervals and repeat the bootstrap with mean lengths five and 60. Forecasts are not refitted within each replicate, so inference is conditional on the fitted walk-forward predictions. Dependence adjustments do not eliminate structural breaks or uncertainty from prior researcher choices. All secondary model and policy comparisons are exploratory. We do not make multiplicity-adjusted discovery claims or interpret a failure to reject as proof of equivalence.

4.5. Target-notional return and cost diagnostic

On date t , let A_t be the assets with OOS forecasts and let $J_t = |A_t|$. For each active asset, set $s_{j,t} = 2\hat{y}_{j,t} - 1$ and target signed weight $w_{j,t} = s_{j,t}/J_t$. Other weights are zero. Gross diagnostic return and one-way target turnover are

$$g_{t+1} = \sum_j w_{j,t} r_{j,t+1}, \quad q_t = \sum_j |w_{j,t} - w_{j,t-1}|, \quad n_{t+1}(c) = g_{t+1} - cq_t. \quad (5)$$

The initial target is zero. Final liquidation adds $\sum_j |w_{j,T}|$ to the final turnover observation. A complete sign reversal costs two units of notional. The reference cost is $c = 0.0009$, interpreted as four basis points of fees plus five of slippage per unit of one-way turnover. These are scenario assumptions, not verified exchange fee schedules or calibrated market-impact estimates.

This is a linear target-notional diagnostic. Turnover ignores position-weight drift and cost-induced changes in sizing; it is not an exact self-financing account. Its close-to-close return window also assumes the day’s closing information is actionable at the boundary without additional latency. The diagnostic is useful for separating signed-return forecasts from turnover costs, but it must not be relabeled as realized execution performance. The portfolio has unit gross exposure when all signals are active; it is directional and need not be market neutral.

Annualized Sharpe is $S = \sqrt{365} \bar{n}/s_n$, with sample standard deviation and zero risk-free rate. We report annualized arithmetic mean $365\bar{n}$ and volatility $\sqrt{365}s_n$ separately. The exact zero-mean cost root within this linear diagnostic is

$$c_{\text{bps}}^* = 10^4 \frac{\bar{g}}{\bar{q}}, \quad \bar{q} > 0. \quad (6)$$

Each bootstrap replicate jointly resamples (g_t, q_t) and recomputes this ratio and net Sharpe. We do not infer the root by interpolating Sharpe or hold turnover fixed when estimating its interval. A negative root means that the resampled gross mean is already negative at zero cost; it is not a feasible negative transaction charge and is not clipped to zero.

4.6. A distinct fixed-horizon execution experiment

To examine an explicit bar-level trading policy, a second calculation enters at O_{t+1} and exits at C_{t+1} every day, including when the following signal has the same direction. It therefore closes and reopens exposure instead of netting repeated target weights. Each asset receives $1/J_t$ of equity measured before any same-day entry. No same-day closing price is used to size another asset’s trade.

For side $s \in \{-1, +1\}$, adverse slippage $\lambda = 0.0005$, and fee rate $f = 0.0004$, define

$$P_{\text{in}} = O_{t+1}(1 + s\lambda), \quad P_{\text{out}} = C_{t+1}(1 - s\lambda), \quad R = s \left(\frac{P_{\text{out}}}{P_{\text{in}}} - 1 \right) - f \left(1 + \frac{P_{\text{out}}}{P_{\text{in}}} \right). \quad (7)$$

The cash ledger independently books signed entry and exit notionals and each leg’s own fee, then checks agreement with this formula. Positions are flat at each daily end. Starting equity is 10,000 accounting units, initial capital is included when calculating the first return, and Sharpe uses only the OOS earning window. There are no entry-bar stop/target assumptions, overlapping two-day returns, or unused pre-evaluation years in this calculation. Instantaneous access to the next bar open remains an idealization: daily data do not identify inference latency, queue priority, or market impact.

5. Results

5.1. Directional accuracy and incremental value

Table 2 reports all classifiers and elementary baselines. Logistic accuracy is 52.95%, while inverse persistence attains 52.88%. The lag-1 ablation and boosted trees do not establish an

improvement over the parameter-free baseline. Similar aggregate accuracies alone would not establish similar information content, since models may disagree on individual observations; the paired results supply the relevant comparison.

Table 2: OOS accuracy (percent). Pooled values are descriptive asset-day averages, not independent-observation significance tests.

Predictor	BTC	ETH	DOGE	Pooled
Logistic	52.71	53.10	53.08	52.95
Boosted trees	52.09	51.37	51.93	51.78
Lag-1 logistic	51.90	52.66	52.58	52.36
Inverse persistence	53.00	52.57	53.15	52.88
Persistence	47.00	47.43	46.85	47.12
Training majority	50.55	49.59	51.72	50.49

Table 3: Paired accuracy differences relative to inverse persistence. Differences and confidence limits are in percentage points; HAC uses 20 lags. The additional model comparisons are exploratory.

Asset	Model	Difference	95% interval	Two-sided p
BTC	Logistic	-0.29	[-2.55, 1.97]	0.803
BTC	Boosted trees	-0.91	[-3.03, 1.21]	0.399
BTC	Lag-1 logistic	-1.10	[-3.31, 1.10]	0.327
ETH	Logistic	+0.53	[-1.85, 2.91]	0.663
ETH	Boosted trees	-1.20	[-3.73, 1.33]	0.353
ETH	Lag-1 logistic	+0.10	[-1.87, 2.06]	0.924
DOGE	Logistic	-0.07	[-2.69, 2.55]	0.957
DOGE	Boosted trees	-1.22	[-3.84, 1.41]	0.363
DOGE	Lag-1 logistic	-0.57	[-1.91, 0.76]	0.399

Per-asset logistic differences are approximately -0.29 , $+0.53$, and -0.07 percentage points for BTC, ETH, and DOGE, respectively (Table 3). Every interval crosses zero. The synchronized date bootstrap gives a pooled difference of 0.07 percentage points, with a 95% interval $[-1.44, 1.61]$. These estimates do not demonstrate incremental superiority. They also do not exclude modest gains or losses; there is no prespecified equivalence margin.

Appendix B reports every fold’s dates, size, and model and baseline accuracy. The final folds have lower accuracy than several earlier folds, but this descriptive pattern is not a formal structural-break finding. Fold sizes differ at the sample end, and averaging fold percentages without weighting would change the estimand.

5.2. Costs and uncertainty

The logistic target-notional diagnostic has gross Sharpe 0.544, with stationary-bootstrap interval $[-0.19, 1.32]$. At nine basis points, net Sharpe is 0.113, with interval $[-0.60, 0.87]$. Mean target turnover is 0.766 per day, implying a cost deduction of 6.89 basis points per day. Annualized net arithmetic mean and volatility are 6.60% and 58.38%, respectively. The annualized mean is not a compounded investment return.

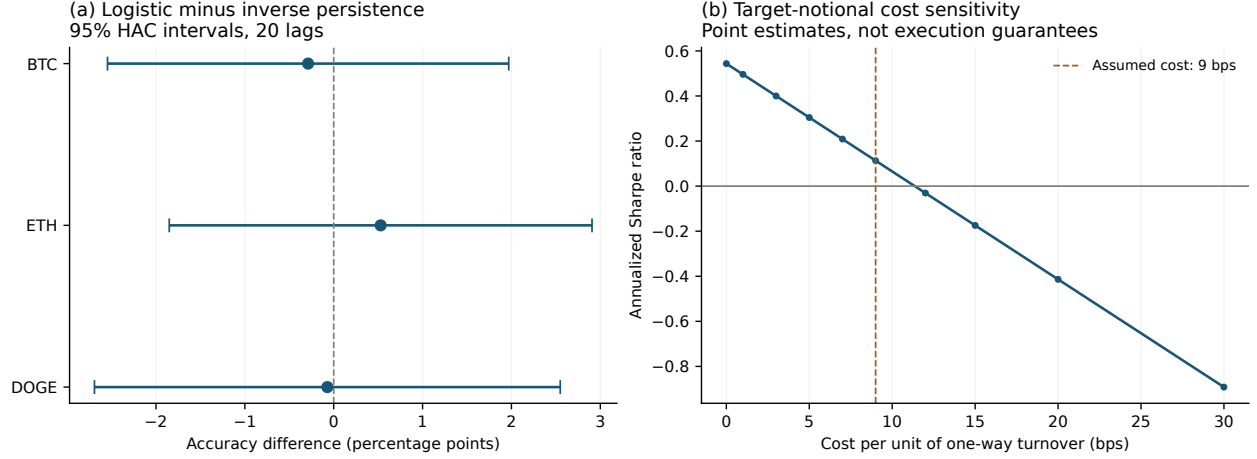


Figure 1: Two distinct evaluation questions. Panel (a) shows paired predictive differences with dependence-aware intervals. Panel (b) shows point-estimate economic sensitivity for the target-notional diagnostic; the curve is not a confidence band or a live execution forecast.

The point-estimate cost root is 11.36 basis points, but its 95% interval spans $[-4.00, 26.43]$. The negative lower bound is economically informative: some resampled paths have negative gross expected return before charging any cost. The width of the interval prevents a precise claim that a particular executable cost level lies safely below break-even. Conversely, an interval crossing zero does not prove that the true expected return is exactly zero.

Table 4: Sensitivity of 95% bootstrap intervals to mean block length. Accuracy differences are percentage points; cost roots are basis points. All variants use 2,000 synchronized replicates.

Block length	Pooled difference	Gross Sharpe	Net Sharpe	Cost root
5	$[-1.49, 1.62]$	$[-0.27, 1.30]$	$[-0.69, 0.86]$	$[-5.68, 26.95]$
20	$[-1.44, 1.61]$	$[-0.19, 1.32]$	$[-0.60, 0.87]$	$[-4.00, 26.43]$
60	$[-1.47, 1.56]$	$[-0.14, 1.30]$	$[-0.56, 0.84]$	$[-3.06, 25.78]$

Table 4 makes the dependence specification visible. These alternatives test sensitivity to the resampling scale, not robustness to every possible form of nonstationarity. The complete deterministic cost curve and gating variants are reported in Appendix C rather than used to select a favorable threshold.

5.3. Policy and benchmark sensitivity

Table 5 gives target-notional return results for all prediction rules, a common three-asset window, and a logistic long/cash variant. The common-window analysis removes the changing asset count from the primary panel, while the long/cash version avoids the assumption of unrestricted short exposure. Neither is treated as a newly selected best policy. All net values in this table still use the approximate target-turnover convention.

Similar hit rates do not imply identical return performance. For example, inverse persistence has gross Sharpe about 0.004 in this diagnostic, compared with 0.544 for logistic regression. Joint date-block resampling gives an annualized arithmetic net-mean advantage for logistic over inverse persistence of 41.17 percentage points, with a 95% interval $[-4.89,$

Table 5: Exploratory target-notional policy comparisons at nine basis points. The common-window calculation starts the portfolio from cash on its own first date.

Variant	Days	Gross SR	Net SR	Turnover
Logistic	2083	+0.544	+0.113	0.766
Boosted trees	2083	+0.234	-0.216	0.770
Lag-1 logistic	2083	+0.386	-0.057	0.817
Inverse persistence	2083	+0.004	-0.547	1.060
Persistence	2083	-0.004	-0.555	1.060
Training majority	2083	+0.112	+0.110	0.002
Logistic, common three-asset dates	1396	+0.947	+0.473	0.755
Logistic, long/cash	2083	+0.964	+0.712	0.383

86.14]. This exploratory economic comparison is uncertain, but it shows why an inconclusive accuracy comparison cannot justify the stronger claim that the model adds no economically relevant information. The positive point estimates for the common-window and long/cash variants also preclude a blanket statement that every tested policy is unprofitable.

The passive reference uses the same available-asset set as the strategy on every date, with positive equal weights. Thus BTC and ETH each receive one half before DOGE becomes eligible, and all three receive one third thereafter. This is a gross rebalanced long-only basket, not a fixed-share buy-and-hold portfolio. Its correlation, beta, and untested intercept relative to the net strategy are exported for descriptive use. We do not infer an absence of alpha from an intercept point estimate or equate a lower raw Sharpe with an invalid diversification role.

The separate daily-liquidation execution experiment produces 5562 round trips and 2083 daily equity returns. Under its stated sizing and cost policy, annualized Sharpe is -0.583 and cumulative return is -94.61%. The magnitude of this loss is not a forecast of live trading performance; it is the outcome of an explicitly costly policy that repurchases or resells the full daily allocation even when signals persist. Daily cash reconciliation error is below 5×10^{-12} accounting units, and final positions are flat. This calculation differs from the target-notional diagnostic in holding, cost realization, timing, and compounding. Their difference cannot be causally assigned to latency or any single software correction.

6. Software verification and audit boundaries

The regression suite for the existing backtest and feature-pipeline components contains 29 tests. It checks next-bar entry scheduling, asset-specific prices, short mark-to-market direction, stop-price behavior on tested bars, hold-expiry contracts, annualization from equity-curve cadence, resampling, cash reconciliation, scaler persistence, and chronological split contracts. The new research checks additionally test causal feature-prefix invariance, known-answer HAC variance, rejection of invalid statistics, joint resampling, benchmark weights, terminal target liquidation, the exact cost root, own-notional fees, saved-model predictions, and fold/result reconciliation.

Mutation testing changes one implementation behavior at a time and reruns the existing 29-test suite. A mutation is considered detected only if the expected named regression test

fails, the total completed test count matches baseline, and the process exit status is consistent. Collection errors, missing tests, and skipped-test runs are not counted as successful detection. Table 6 reports eight selected mutants, all detected.

Table 6: Selected software mutations. Counts refer to failed tests among the baseline 29. The eight mutants are deliberately chosen, not a random sample of possible defects.

Introduced behavior	Failed tests	Outcome
Future-bar price lookup	1	Detected
Wrong short valuation sign	1	Detected
Close-only stop checks	2	Detected
Incorrect Sharpe annualization	2	Detected
Permutation used as resampling	1	Detected
Refit scaler during inference	2	Detected
Remove specified split gap	2	Detected
Split fees equally across legs	1	Detected

This result does not imply complete branch coverage, that all historical faults were independently reproduced, or that eight tests establish economic validity. In particular, the split-gap mutation verifies a specified contract; it does not prove that every split without a five-row gap necessarily leaks information. Similarly, an execution fixture cannot determine whether an external feature was observable at a forecast timestamp.

The supplied draft also described randomized engine tests as a calibrated null experiment. We do not use that claim as empirical evidence here: its 2,400 fifteen-minute bars represent 25 days per path, not 75, and exponentiating zero-mean Gaussian increments gives zero log drift rather than zero expected simple return. A distributional diagnostic would additionally need consistent risk-free-rate treatment and terminal liquidation. Historical extreme Sharpe values and a twelve-fault narrative are therefore not used to make a quantitative causal attribution in this revision. The manuscript’s verified engineering claims are restricted to the executable tests and selected mutations actually supplied and rerun.

7. Discussion and limitations

The central lesson concerns the estimand. Accuracy measures whether a direction is correct; paired accuracy measures whether a model adds to a baseline; a signed-return series measures magnitude-sensitive performance; a cash ledger measures the consequences of a particular trade policy. These objects can disagree without contradiction. Here, a seemingly attractive aggregate hit rate is closely matched by inverse persistence, and a positive gross Sharpe is accompanied by substantial uncertainty and cost sensitivity.

The study supports several practical reporting choices. Prediction records should carry decision dates, label dates, fold identifiers, probabilities, and baseline decisions. Preprocessing must be part of the saved model rather than an unspecified external step. Cost analysis should report the convention for one-way versus round-trip turnover and include initial and terminal actions. Finally, software tests should publish their exact scope and expected failure behavior. These recommendations follow from this artefact’s measurement problems, not from a claim that the same problems affect every published study.

The limits are material. The evidence concerns three retrospectively chosen surviving assets at one exchange and a UTC daily boundary. It does not address delisted assets, selection bias across a broader universe, other exchanges, or re-anchored bars. Public archive spot checks increase confidence in selected inputs, but are not a full-history verification or proof against exchange revisions. The 30 features omit order books, funding, basis, open interest, liquidations, cross-asset information, and independently timestamped text.

The models are modest fixed specifications rather than an exhaustive comparison with modern architectures. Logistic probabilities are not externally calibrated. Optimizing classification loss does not optimize a turnover-aware investment objective. The OOS folds remain retrospective because the source analysis was inspected before this revision, and an untouched prospective validation period is still needed before any deployment claim.

HAC and the stationary bootstrap address selected dependence structures, not arbitrary regime shifts. The bootstrap does not refit the model or model estimation uncertainty under alternative research paths. Wide confidence intervals limit claims of both superiority and inferiority. Secondary models, gates, and portfolio variants are exploratory; no familywise discovery guarantee is provided.

Economically, spot prices do not make shorting automatically feasible. Borrowing constraints, margin requirements, liquidation, financing, funding, capacity, and size-dependent impact are absent. The long/cash diagnostic is a useful sensitivity but still not a live implementation. The target-notional approach omits weight drift, and the daily-liquidation ledger uses idealized bar-boundary fills. No measured exchange latency, paper trading, or broker/exchange reconciliation is available. None of the reported ratios should be interpreted as an expected investor return.

8. Conclusion

This reproducible audit evaluates a 30-feature daily cryptocurrency forecasting implementation using explicit chronological folds, elementary baselines, dependent-data uncertainty, and auditable economic conventions. Logistic regression achieves 52.95% accuracy over 5562 OOS asset-days, compared with 52.88% for inverse persistence. The paired difference is small and uncertain, and the result does not establish incremental superiority. A gross Sharpe of 0.544 falls to 0.113 under the reference target-turnover cost scenario, while a distinct daily-liquidation policy has materially different outcomes. Eight selected software mutations are detected, with the coverage boundary stated explicitly. The contribution is a transparent research artefact and a carefully bounded negative or inconclusive result, not a new universally superior forecasting model or a proven deployable strategy.

Data and code availability

The local replication package contains the frozen input files, source code, fitted fold pipelines, dated predictions, trade ledger, machine-readable results, checksums, and reproduction instructions. The manuscript’s numeric macros and tables are generated from these results. No public DOI or archival deposit has yet been assigned. Authors must confirm redistribution rights and arrange an anonymized reviewer supplement or an appropriate repository deposit before submission. No public release is implied by preparation of the local package.

Funding

Author completion required: Confirm funding sources and funder roles, or supply an accurate no-funding statement.

Declaration of competing interests

Author completion required: All authors must confirm competing interests and complete the publisher’s required declaration.

Ethics and consent

The reported empirical analysis uses market-price and trading-activity bars, not human-participant records or personal social-media data. No human-participant experiment is described. Authors should confirm any institutional requirements applicable to their work.

Declaration of generative AI and AI-assisted technologies in the writing process

OpenAI Codex was used to assist with manuscript drafting and revision. AI assistance was also used to develop and check research code; this is disclosed separately from the writing use.

Author completion required: The human authors must independently review the methods, code, numerical outputs, references, and wording; verify compliance with the journal’s AI policy; and finalize a truthful disclosure and accountability statement. This draft does not assert that this review has already occurred.

Appendix A. Complete feature definitions

Let O, H, L, C, V, Q, N, B denote open, high, low, close, base volume, quote volume, trade count, and taker-buy base volume. Let μ_k and σ_k be trailing k -bar mean and sample standard deviation (ddof = 1), and $z_k(X) = (X - \mu_k(X))/\sigma_k(X)$. Every rolling window ends at t . Undefined ratios become missing and are removed before fitting.

Table A.7: Executable feature inventory. Counts sum to 30.

Group	Count	Definitions
Log returns	6	$\log(C_t/C_{t-k})$, $k \in \{1, 2, 3, 5, 10, 20\}$.
Volatility/location	8	$\sigma_k(\log(C_t/C_{t-1}))$ and $z_k(C_t)$, $k \in \{5, 10, 20, 60\}$.
Bar shape	3	$(H - L)/C$, $(C - O)/O$, $(C - L)/(H - L)$.
Trading activity	7	B/V , $\mu_5(B/V)$, Q/N , $z_{60}(Q/N)$, $z_{60}(N)$, $z_{60}(V)$, $V/\mu_{20}(V)$.
Technical/calendar	6	RSI, MACD histogram, Bollinger position, normalized ATR, sine and cosine of weekday, defined below.

With $\delta_t = C_t - C_{t-1}$, RSI is $100 - 100/[1 + \mu_{14}(\max(\delta, 0))/\mu_{14}(\max(-\delta, 0))]$. This uses simple rolling averages, not Wilder’s recursive smoothing. The MACD histogram is $M - \text{EMA}_9(M)$, where $M = \text{EMA}_{12}(C) - \text{EMA}_{26}(C)$. Exponential means use $\alpha = 2/(k + 1)$, recursive initialization at the first observation, and `adjust=False`. Bollinger position is $(C - \mu_{20}(C))/(2\sigma_{20}(C))$. True range is $\max\{H - L, |H - C_{t-1}|, |L - C_{t-1}|\}$; normalized ATR is its simple 14-bar mean divided by C . Calendar features are $\sin(2\pi d/7)$ and $\cos(2\pi d/7)$ for weekday $d = 0, \dots, 6$. The fields summarize daily exchange activity; they are not order-book snapshots.

Appendix B. Complete fold results

Table B.8: Fold-level OOS accuracy in percent; L denotes logistic and I inverse persistence. Dates are prediction origins.

Asset	Fold	Start	End	n	L	I
BTC	1	2020-07-17	2021-03-23	250	50.0	55.6
BTC	2	2021-03-24	2021-11-28	250	50.4	55.6
BTC	3	2021-11-29	2022-08-05	250	51.6	50.8
BTC	4	2022-08-06	2023-04-12	250	51.6	54.0
BTC	5	2023-04-13	2023-12-18	250	54.8	54.8
BTC	6	2023-12-19	2024-08-24	250	59.2	54.8
BTC	7	2024-08-25	2025-05-01	250	55.2	52.0
BTC	8	2025-05-02	2026-01-06	250	52.0	50.0
BTC	9	2026-01-07	2026-03-30	83	43.4	42.2
ETH	1	2020-07-17	2021-03-23	250	49.2	52.0
ETH	2	2021-03-24	2021-11-28	250	54.4	52.4
ETH	3	2021-11-29	2022-08-05	250	48.0	50.8
ETH	4	2022-08-06	2023-04-12	250	50.0	54.8
ETH	5	2023-04-13	2023-12-18	250	61.6	54.0
ETH	6	2023-12-19	2024-08-24	250	54.4	56.0
ETH	7	2024-08-25	2025-05-01	250	57.6	54.0
ETH	8	2025-05-02	2026-01-06	250	53.2	48.4
ETH	9	2026-01-07	2026-03-30	83	42.2	47.0
DOGE	1	2022-06-04	2023-02-08	250	55.6	58.8
DOGE	2	2023-02-09	2023-10-16	250	58.0	57.6
DOGE	3	2023-10-17	2024-06-22	250	52.8	53.6
DOGE	4	2024-06-23	2025-02-27	250	52.4	51.2
DOGE	5	2025-02-28	2025-11-04	250	48.8	49.2
DOGE	6	2025-11-05	2026-03-30	146	49.3	45.2

Appendix C. Additional cost and gating results

Table C.9: Target-notional logistic cost curve. The daily mean is net of the stated turnover charge, expressed in basis points.

One-way cost (bps)	Mean daily return (bps)	Annualized SR
0	+8.70	+0.544
1	+7.93	+0.496
3	+6.40	+0.400
5	+4.87	+0.304
7	+3.34	+0.209
9	+1.81	+0.113
12	-0.49	-0.031
15	-2.79	-0.174
20	-6.62	-0.414
30	-14.27	-0.892

Table C.10: Confidence gating at nine basis points. An asset retains its original $1/J_t$ allocation only when $|p - 0.5| \geq h$; excluded allocations remain cash. No threshold is selected from these results.

Threshold h	Net annualized SR	Target turnover
0.00	+0.113	0.766
0.02	+0.018	0.736
0.04	-0.080	0.642
0.06	-0.191	0.522
0.08	-0.309	0.388

Gating reduces turnover, but the reported point estimates do not identify a superior threshold. These exploratory results are not a formal calibration test and do not establish that model confidence is intrinsically anti-informative.

Appendix D. Replication instructions and integrity checks

The primary command is `python scripts/reproduce_jfds.py` after installing `requirements-research.txt` under Python 3.12. It rebuilds the predictions and models, runs the focused regression and mutation suites, and regenerates tables and figures. An optional network command, `python scripts/verify_jfds_sources.py`, repeats the selected-month source check without replacing local inputs. The replication does not need database, exchange-account, social-media, or language-model credentials.

Saved model files use Python’s joblib/pickle format and must only be loaded from trusted sources. The package includes hash manifests to identify the local artefact; these hashes establish file identity, not a claim of copyright ownership or independent scientific validation. Reproduction records dependency versions and test output. Tables are generated from machine-readable outputs, reducing opportunities for manual transcription errors. Author

details and declarations remain outside the empirical calculation and must be completed by the human authors.

References

- Arnott, R., Harvey, C.R., Markowitz, H., 2019. A backtesting protocol in the era of machine learning. *The Journal of Financial Data Science* 1, 64–74. doi:10.3905/jfds.2019.1.064.
- Binance, 2026. Binance public data. URL: <https://github.com/binance/binance-public-data>. documentation accessed 22 September 2026.
- Diebold, F.X., Mariano, R.S., 1995. Comparing predictive accuracy. *Journal of Business & Economic Statistics* 13, 253–263. doi:10.1080/07350015.1995.10524599.
- Gradojevic, N., Kukolj, D., Adcock, R., Djakovic, V., 2023. Forecasting bitcoin with technical analysis: A not-so-random forest? *International Journal of Forecasting* 39, 1–17. doi:10.1016/j.ijforecast.2021.08.001.
- Jaquart, P., Dann, D., Weinhardt, C., 2021. Short-term bitcoin market prediction via machine learning. *The Journal of Finance and Data Science* 7, 45–66. doi:10.1016/j.jfds.2021.03.001.
- Jia, Y., Harman, M., 2011. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering* 37, 649–678. doi:10.1109/TSE.2010.62.
- Kaufman, S., Rosset, S., Perlich, C., Stitelman, O., 2012. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data* 6, 15:1–15:21. doi:10.1145/2382577.2382579.
- Newey, W.K., West, K.D., 1987. A simple, positive semi-definite, heteroskedasticity and autocorrelation consistent covariance matrix. *Econometrica* 55, 703–708. doi:10.2307/1913610.
- Politis, D.N., Romano, J.P., 1994. The stationary bootstrap. *Journal of the American Statistical Association* 89, 1303–1313. doi:10.1080/01621459.1994.10476870.